Chaos Gameday Executive Report

Resilience validation of the Banking workloads on Kubernetes infrastructure PM_Demo/hpbprimary Account h1YZZt7mSo-_6odvC8wbEA | Org PMDemo | Project PM_Demo | Report date 2026-10-28

2	1	1	50%
Experiments assessed	Latest runs passed	Latest runs failed	Average resilience score

1. Executive Summary

Two chaos experiments were assessed using the most recent run of each. The **pod-delete** experiment on the frontend-service **passed with a 100% resilience score**: the account-validation endpoint kept returning HTTP 200 on all 10 probe iterations. The **pod-network-latency** experiment on the account-service **failed with a 0% resilience score**: the payment settlement-rate probe measured 0 against a required minimum of 10.

- **Strength:** the frontend-service tolerates the loss of a pod with no impact on account validation.
- **Risk:** the payment settlement path did not meet its success criteria while the latency experiment ran. The cause is not yet established (see section 4), but it needs triage before this path is called resilient.
- **Gap:** both experiments ran once under a short chaos window and are not scheduled (no cron). Neither provides continuous assurance.

2. Results at a Glance

Experiment	Fault / Target	#	Executed by	Executed (UTC)	Dur.	Score	Outcome
test	pod-delete deployment/frontend-service (banking)	#1	pritesh.kiri@harness.io	2026-05-13 08:35:54	2m 3s	100%	PASSED
transaction-settlement-delay	pod-network-latency (2s) deployment/account-service (banking)	#2	udit.gaurav@harness.io	2026-04-08 09:37:05	58s	0%	FAILED (probe failure)

Scores and outcomes come from the v3 chaos-experiment-run report for each experiment's latest run. Resilience score is the weighted share of passed probes.

3. Experiment Activity: What, When and Who

Experiment	Experiment owner (created by)	Created (UTC)	Total runs	Last run	Schedule
test (a428f066-e6d4-4afe-8de6-3baa760a4583)	pritesh.kiri@harness.io (Account Viewer)	2026-05-13 08:35	1	2026-05-13 08:35:54 UTC	Not scheduled
transaction-settlement-delay (023843af-10ca-4d99-b364-b4d7017298d4)	udit.gaurav@harness.io (Account Admin)	2026-04-08 09:33	2	2026-04-08 09:37:05 UTC	Not scheduled

Run history for transaction-settlement-delay

#	Started (UTC)	Status	Score	Note
#1	2026-04-08 09:34:26	Error	n/a	Run did not complete, so no resilience verdict was produced.
#2	2026-04-08 09:37:05	Completed with probe failure	0%	Latest run; analysed in this report.

4. Detailed Findings

4.1 test: pod-delete on frontend-service (PASSED, 100%)

Item	Detail
Run ID	1cc13483-12e3-49ec-9301-61a687bc2135 (Run #1)
Window	2026-05-13 08:35:54 to 08:37:57 UTC (2m 3s)
Infrastructure	PM_Demo/hpbprimary (KubernetesV2), environment PM_Demo
Fault	pod-delete, status Completed, 52.38s. Target: deployment frontend-service, namespace banking, label app=validator-service. Chaos duration 30s, interval 10s, 0% of pods affected parameter (default), sequence parallel.
Probe	hpb-get-account-validation (httpProbe), status **Passed**. GET http://api.ejbnrunsci.com:8080/api/v1/accounts/user/4, expected HTTP 200, got 200. Weightage 10.
Probe iterations	10 of 10 passed, each about 1.04 to 1.10 seconds.

Interpretation: the account-validation API stayed available throughout the pod deletion with consistent latency near 1 second. **Caveat:** the fault targets frontend-service while its label selector reads app=validator-service; confirm the selector hits the pods intended for the test, otherwise the pass may be weaker evidence than it appears.

4.2 transaction-settlement-delay: pod-network-latency on account-service (FAILED, 0%)

Item	Detail
Run ID	3ce4ba56-29cb-44fe-8ee5-183d6f217b33 (Run #2)
Window	2026-04-08 09:37:05 to 09:38:03 UTC (58s)
Infrastructure	PM_Demo/hpbprimary (KubernetesV2), environment PM_Demo
Fault	pod-network-latency, status Completed, 51.04s (09:37:07 to 09:37:58). Target: deployment account-service, namespace banking. NETWORK_LATENCY=2s, TOTAL_CHAOS_DURATION=30, interface eth0, containerd runtime.
Probe	payment-settlement-rate-validation (promProbe), status **Failed**, 5.66s (09:37:06 to 09:37:12). Query: sum(rate(account_operation_total{operation='update_balance',status='success'}[2m])). Criteria: value >= 10. Weightage 10.
Probe result	Actual value 0; required at least 10. 1 iteration, failed.

Interpretation: the successful balance-update rate was 0 per second against a threshold of 10. Two readings are possible and the report cannot separate them:

- **Genuine resilience gap:** 2 seconds of added latency on account-service stops balance updates from completing successfully.
- **Invalid test conditions:** the probe began at 09:37:06, one second before the fault started at 09:37:07, and finished at 09:37:12. The 2-minute rate window was therefore sampled almost entirely before the fault could take effect. A value of 0 at that moment points to no baseline traffic or a missing metric, not to latency impact.

The second reading is the more likely one on the timing evidence, but it is a hypothesis to verify, not a conclusion.

5. Recommendations

#	Action	Why	Priority
1	Re-run transaction-settlement-delay with steady synthetic load on update_balance, and confirm the Prometheus query returns a baseline of at least 10 before injecting latency.	Separates a real gap from an invalid test.	**High**
2	Move the settlement probe to run continuously during the fault (not at the start only), or add a pre-chaos baseline probe.	The current probe timing barely overlaps the fault.	**High**
3	Investigate why run #1 of transaction-settlement-delay ended in Error.	An unexplained error undermines confidence in the pipeline.	**Medium**
4	Confirm the label selector for the pod-delete test (app=validator-service vs frontend-service).	Ensures the 100% score reflects the intended target.	**Medium**
5	Schedule both experiments (for example weekly) and extend chaos duration beyond 30s.	Turns one-off checks into continuous assurance.	**Medium**

6. Methodology and Data Sources

- Experiment IDs were read from the environment and the latest run was taken from the v2 experiment API (RecentExperimentRunDetails, highest run sequence).
- Run results were taken from the v3 API: /gateway/chaos/manager/api/v3/chaos-experiment-run/report/{experimentId}/{chaosExperimentRunId}.
- User names were resolved from the Harness NG user API. Experiment owner is the creator of the experiment; the run executor is as stated in each run report.
- Only the latest run per experiment is analysed. Historical trends are not covered.